SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA0306	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Y SRAVAN KUMAR	**Reg. No.:**	192472289
Section / Batch:	CSE AI 2024	**Date:**	18-08-2026

Code of Conduct

I __Y SRAVAN KUMAR (192472289) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____SRAVAN_____

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- C(machine) = 3
- C(machine learning) = 3
- C(learning improves) = 1
- C(learning enables) = 1
- C(learning drives) = 1
- C(learning improves business) = 1

The interpolation weights are:

- λ_1 = 0.5 (Trigram)
- λ_2 = 0.3 (Bigram)
- λ_3 = 0.2 (Unigram)

The prediction probabilities after the word **"learning"** are:

- P(improves) = 0.33
- P(enables) = 0.33
- P(drives) = 0.33

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **P(learning | machine)**. Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **P(improves) = 0.33, P(enables) = 0.33, P(drives) = 0.33**. Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- P(book | VB) = 0.7
- P(book | NN) = 0.3
- P(Start → VB) = 0.6
- P(Start → NN) = 0.4

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	18	18	15	15	3	69	73
2	17	18	14	15	3	67	
3	18	17	15	14	3	67	

Faculty In-charge	Course Coordinator	Program Director
Signature: _Dr.T.Kumaragurubaran_____	Signature: _____	Signature: _____
Date: ____18-08-2026_____	Date: _____	Date: _____

Case Study 1 – E-Commerce Smart Search and Product Prediction System

1. MLE of Bigram Probability

Formula:

P(learning machine)=C(machine)C (machine learning)

Given:

C(machine learning)=3

C(machine)=3

Therefore:

P(learning|machine)=33 =1.0

Interpretation

The probability is 100%, meaning every occurrence of "machine" in the training corpus is followed by "learning". Therefore, an e-commerce search system would strongly predict "learning" after the user types "machine".

2. Backoff Model – "machine learning transforms"

The sequence "machine learning transforms" is unseen.

For the trigram:

P(transforms|machine,learning)=0

because:

C(machine learning transforms)=0

Back off to the bigram:

P(transforms|learning)=C(learning)C(learning transforms) =30 =0

Finally, back off to the unigram:

P(transforms)=12C(transforms) =0

Therefore:

P(transforms)=0

Why Backoff is useful

It handles unseen N-gram sequences.

It uses lower-order information when higher-order information is unavailable.

It makes prediction more robust for new customer search queries.

In this particular corpus, "transforms" does not occur at all, so even the unigram probability is 0. In a practical search system, smoothing or an unknown-word token would normally be added to avoid this.

3. Deleted Interpolation

We combine:

P(w|machine,learning)=λ1 Ptri +λ2 Pbi +λ3 Puni

For "machine learning improves":

Trigram

P(improves|machine,learning)=C(machine learning)C(machine learning improves) =31 =0.3333

Bigram

P(improves|learning)=C(learning)C(learning improves) =31 =0.3333

Unigram

Total words = 12 and improves occurs once:

$P(\text{improves}) = \frac{1}{12} = 0.0833$

Using:

$\lambda_1 = 0.5$

$\lambda_2 = 0.3$

$\lambda_3 = 0.2$

$P = 0.5(0.3333) + 0.3(0.3333) + 0.2(0.0833)$ $P = 0.1667 + 0.1000 + 0.0167$

$P(\text{improves} \mid \text{machine, learning}) = 0.2834$

Interpretation

Interpolation combines information from trigram, bigram, and unigram models, making predictions more reliable when some N-grams have limited training examples.

4. Entropy

Given:

$P(\text{improves}) = 0.33$

$P(\text{enables}) = 0.33$

$P(\text{drives}) = 0.33$

Formula:

$H = -\sum P(x) \log_2 P(x)$ $H = -3(0.33 \log_2 (0.33))$ $H \approx 1.58$ bits

The theoretical maximum for 3 equally likely outcomes is:

$\log_2 (3) \approx 1.585$

Interpretation

The entropy is high and close to the maximum, indicating that the system has high uncertainty about the next word. All three words have almost equal probability, so the search system cannot confidently choose one prediction.

Implementation

NLP CLASS EXP 3.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP CLASS EXP 3.py (3.12.3)

File Edit Format Run Options Window Help

```
import nltk
import numpy as np
import math
from collections import Counter
from nltk.util import ngrams

# Corpus
text = """machine learning improves business,
machine learning enables automation,
machine learning drives innovation"""

# Tokenization
words = nltk.word_tokenize(text.lower())

# N-grams
unigram = Counter(words)
bigram = Counter(ngrams(words, 2))
trigram = Counter(ngrams(words, 3))

total = len(words)

print("----- N-GRAM COUNTS -----")
print("Unigram:", unigram)
print("Bigram:", bigram)
print("Trigram:", trigram)

# 1. MLE Bigram
p_bigram = bigram[("machine", "learning")] / unigram["machine"]

print("\n----- MLE BIGRAM -----")
print("P(learning | machine) =", p_bigram)

# 2. Backoff
word = "transforms"

p_tri = trigram[("machine", "learning", word)] / bigram[("machine", "learning")]

if p_tri == 0:
    p_bi = bigram[("learning", word)] / unigram["learning"]

    if p_bi == 0:
        p_backoff = unigram[word] / total
    else:
        p_backoff = p_bi
else:
```

Output

IDLE Shell 3.12.3

File Edit Shell Debug Options Window Help

```
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP CLASS EXP 3.py
----- N-GRAM COUNTS -----
Unigram: Counter({'machine': 3, 'learning': 3, ',': 2, 'improves': 1, 'business': 1, 'enables': 1, 'automation': 1, 'drives': 1, 'innovation': 1})
Bigram: Counter({('machine', 'learning'): 3, (',', 'machine'): 2, ('learning', 'improves'): 1, ('improves', 'business'): 1, ('business', ','): 1, ('learn
ing', 'enables'): 1, ('enables', 'automation'): 1, ('automation', ','): 1, ('learning', 'drives'): 1, ('drives', 'innovation'): 1})
Trigram: Counter({(',', 'machine', 'learning'): 2, ('machine', 'learning', 'improves'): 1, ('learning', 'improves', 'business'): 1, ('improves', 'busines
s', ','): 1, ('business', ',', 'machine'): 1, ('machine', 'learning', 'enables'): 1, ('learning', 'enables', 'automation'): 1, ('enables', 'automation',
','): 1, ('automation', ',', 'machine'): 1, ('machine', 'learning', 'drives'): 1, ('learning', 'drives', 'innovation'): 1})

----- MLE BIGRAM -----
P(learning | machine) = 1.0

----- BACKOFF -----
P(transforms | machine, learning) = 0.0
P(transforms | learning) = 0.0
P(transforms) = 0.0
Final Backoff Probability = 0.0

----- DELETED INTERPOLATION -----
Trigram = 0.3333333333333333
Bigram = 0.3333333333333333
Unigram = 0.07142857142857142
Interpolated Probability = 0.28095238095238095

----- ENTROPY -----
Entropy = 1.5834674497121084 bits

----- FINAL PREDICTION -----
Next Word = improves
>>>
```

CASE STUDY 2 – AI-Powered Hospital Appointment Chatbot

1. Penn Treebank POS Tagging

Sentence 1

"Book an appointment with the doctor."

Word	POS Tag	Meaning
Book	VB	Verb
an	DT	Determiner
appointment	NN	Noun
with	IN	Preposition
the	DT	Determiner
doctor	NN	Noun
.	.	Punctuation

Book = VB because it represents an action/request.

Sentence 2

"The book contains medical information."

Word	POS Tag	Meaning
The	DT	Determiner
book	NN	Noun
contains	VBZ	Verb
medical	JJ	Adjective
information	NN	Noun
.	.	Punctuation

Book = NN because it refers to a physical/document object.

2. HMM Probability Calculation

Given:

$P(book|VB)=0.7$

$P(book|NN)=0.3$

$P(Start \rightarrow VB)=0.6$

$P(Start \rightarrow NN)=0.4$

Verb

$P(VB,book)=P(VB|Start)\times P(book|VB) =0.6\times 0.7=0.42$

Noun

$P(NN,book)=P(NN|Start)\times P(book|NN) =0.4\times 0.3=0.12$

Result

0.42>0.12

Therefore, the HMM predicts:

book → VB

The model favors VB because its combined transition and emission probability is higher.

3. Rule-Based vs HMM Tagging

Feature	Rule-Based	HMM
Method	Hand-written grammar rules	Probability-based
Context	Uses linguistic rules	Uses learned probabilities
Accuracy	Good for known patterns	Good with sufficient training data
Ambiguity	Depends on rules	Handles ambiguity probabilistically
Training	No training required	Requires training data
Limitation	Rules can become complex	Depends on quality of data

Healthcare chatbot: HMM is useful for handling different sentence structures, while rules are useful for important predefined medical commands.

4. Importance of Penn Treebank POS Tags

Intent Detection: Helps identify actions such as book, cancel, and check.

Entity Identification: Helps identify nouns such as doctor, appointment, and hospital.

Response Generation: Helps understand sentence structure and generate appropriate responses.

Standardization: Provides a consistent representation of grammatical information for NLP models.

CODE

nlp class exp 4.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/nlp class exp 4.py (3.12.3)

File Edit Format Run Options Window Help

```
import nltk

# Sentences
s1 = "Book an appointment with the doctor."
s2 = "The book contains medical information."

# Tokenization and POS tagging
for sentence in [s1, s2]:
    tokens = nltk.word_tokenize(sentence)
    tags = nltk.pos_tag(tokens)

    print("\nSentence:", sentence)
    print("Tokens:", tokens)
    print("POS Tags:", tags)

# HMM probabilities
emission = {"VB": 0.7, "NN": 0.3}
transition = {"VB": 0.6, "NN": 0.4}

p_vb = transition["VB"] * emission["VB"]
p_nn = transition["NN"] * emission["NN"]

print("\n--- HMM Calculation ---")
print("P(VB) =", transition["VB"], "*", emission["VB"], "=", p_vb)
print("P(NN) =", transition["NN"], "*", emission["NN"], "=", p_nn)

if p_vb > p_nn:
    print("Most probable tag for 'book': VB")
else:
    print("Most probable tag for 'book': NN")
```

OUTPUT

IDLE Shell 3.12.3

File Edit Shell Debug Options Window Help

```
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> 
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/nlp class exp 4.py

Sentence: Book an appointment with the doctor.
Tokens: ['Book', 'an', 'appointment', 'with', 'the', 'doctor', '.']
POS Tags: [('Book', 'NNP'), ('an', 'DT'), ('appointment', 'NN'), ('with', 'IN'), ('the', 'DT'), ('doctor', 'NN'), ('.', '.')]

Sentence: The book contains medical information.
Tokens: ['The', 'book', 'contains', 'medical', 'information', '.']
POS Tags: [('The', 'DT'), ('book', 'NN'), ('contains', 'VBZ'), ('medical', 'JJ'), ('information', 'NN'), ('.', '.')]

--- HMM Calculation ---
P(VB) = 0.6 * 0.7 = 0.42
P(NN) = 0.4 * 0.3 = 0.12
Most probable tag for 'book': VB
>>> 
```

CASE STUDY 3 – Financial News POS Tag Correction and Uncertainty Analysis

1. Transformation Rule

Initial tags:

market/NN growth/NN drives/NNS investment/NN

Rule:

Change NNS → VBZ if the preceding word is tagged NN.

Since growth is tagged **NN** and drives is tagged **NNS**:

drives/NNS → drives/VBZ

Corrected POS Tags

market/NN growth/NN drives/VBZ investment/NN

Explanation: drives is the main verb of the sentence, and it agrees with the singular subject growth, so VBZ is appropriate.

2. Tagging Error of "drives"

Initial tag: NNS (plural noun)

Correct tag: VBZ (third-person singular verb)

growth acts as the singular subject.

drives is the action performed by the subject.

Therefore:

drives/VBZ

3. Relative Frequency

Total frequency:

500+350+180+420=1450

Word	Frequency	Probability
market	500	0.3448
growth	350	0.2414
drives	180	0.1241
investment	420	0.2897

Formula:

P(word)=Total FrequencyFrequency(word)

Importance

Frequent words have higher probabilities.

Corpus statistics help statistical POS taggers make predictions.

Frequency information helps NLP systems identify common language patterns.

4. Entropy

Formula:

$H=-\sum P(x)\log_2 P(x)$

Using the four probabilities:

H≈1.916 bits

Before and After Transformation

The transformation only changes the POS tag of drives. It does not change the word frequencies.

Therefore:

Before transformation = 1.916 bits

After transformation = 1.916 bits

Interpretation: Entropy measures uncertainty in the word-frequency distribution. Since the word frequencies remain unchanged, the entropy also remains unchanged.

CODE

NLP CLASS EXP 5.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP CLASS EXP 5.py (3.12.3)

File Edit Format Run Options Window Help

```
# Tokenization
words = nltk.word_tokenize(sentence)

# Initial POS tagging
initial = nltk.pos_tag(words)

print("Initial POS Tags:")
print(initial)

# Apply transformation rule
corrected = initial.copy()

for i in range(1, len(corrected)):
    if corrected[i][1] == "NNS" and corrected[i-1][1] == "NN":
        corrected[i] = (corrected[i][0], "VBZ")

print("\nCorrected POS Tags:")
print(corrected)

# Corpus frequencies
freq = {
    "market": 500,
    "growth": 350,
    "drives": 180,
    "investment": 420
}

total = sum(freq.values())

print("\nFrequency Distribution:")
for word, count in freq.items():
    print(word, ":", count, "Probability:", round(count/total, 4))

# Entropy
prob = [count/total for count in freq.values()]
entropy = -sum(p * math.log2(p) for p in prob)

print("\nEntropy Before Transformation:", round(entropy, 3), "bits")
print("Entropy After Transformation :", round(entropy, 3), "bits")

print("\nFinal POS Tagging:")
for word, tag in corrected:
    print(word, "/", tag)
```

OUTPUT

IDLE Shell 3.12.3

File Edit Shell Debug Options Window Help

```
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/NLP CLASS EXP 5.py
Initial POS Tags:
[('Market', 'NN'), ('growth', 'NN'), ('drives', 'VBZ'), ('investment', 'NN'), ('.', '.')]

Corrected POS Tags:
[('Market', 'NN'), ('growth', 'NN'), ('drives', 'VBZ'), ('investment', 'NN'), ('.', '.')]

Frequency Distribution:
market : 500 Probability: 0.3448
growth : 350 Probability: 0.2414
drives : 180 Probability: 0.1241
investment : 420 Probability: 0.2897

Entropy Before Transformation: 1.916 bits
Entropy After Transformation : 1.916 bits

Final POS Tagging:
Market / NN
growth / NN
drives / VBZ
investment / NN
. / .
>>>
```